

Improved Density Peaks Clustering (DPC) Algorithm

Xiuyuan Lu

May 4, 2025

1 Algorithm Overview

The improved DPC algorithm enhances the original method by Rodriguez and Laio (2014) through automated center selection, KNN-based density estimation, and noise reassignment. Key steps are outlined below.

2 Step-by-Step Breakdown

2.1 Data Preprocessing

- **Purpose:** Reduce dimensionality for visualization while clustering in original space.

- **Equation:**

$$\mathbf{X}_{\text{PCA}} = \text{PCA}(\mathbf{X}, \text{n_components} = 2) \quad (1)$$

- **Implementation:**

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(data) # For visualization only
```

2.2 Density Estimation (KNN-based)

- **Original DPC:**

$$\rho_i = \sum_j \chi(d_{ij} - d_c) \quad (2)$$

- **Improved Equation:**

$$\rho_i = \frac{1}{\text{mean}(d_{i,\text{KNN}})} \quad (3)$$

- **Code:**

```

k = 5
nbrs = NearestNeighbors(n_neighbors=k).fit(data)
knn_distances, _ = nbrs.kneighbors(data)
density = 1 / (np.mean(knn_distances, axis=1) + 1e-10)

```

2.3 Delta Calculation

- **Equation:**

$$\delta_i = \begin{cases} \min_{j: \rho_j > \rho_i} (d_{ij}) & \text{if } \exists j \text{ s.t. } \rho_j > \rho_i \\ \max_j (d_{ij}) & \text{otherwise} \end{cases} \quad (4)$$

- **Implementation:**

```

distances = squareform(pdist(data))
delta = np.zeros_like(density)
for i in range(len(data)):
    higher_density = density > density[i]
    delta[i] = np.min(distances[i, higher_density])
    if higher_density.any() else np.max(distances[i])

```

2.4 Automated Center Selection

- **Metric:**

$$\gamma_i = \rho_i \times \delta_i \quad (5)$$

- **Selection:**

1. Sort γ_i in descending order.
2. Use `KneeLocator` to find the elbow point.
3. Enforce cluster count within a target range (e.g., 8–9).

- **Code:**

```

gamma = density * delta
n_clusters = enforce_cluster_count(gamma) # Knee detection
centers = np.argsort(-gamma)[:n_clusters]

```

2.5 Cluster Assignment (K-means Initialization)

- **Improvement:** Replace original single-step assignment with K-means initialized at selected centers.
- **Code:**

```
kmeans = KMeans(n_clusters=n_clusters, init=data.iloc[centers],
                 n_init=1, random_state=42)
dpc_labels = kmeans.fit_predict(data)
```

2.6 Noise Reassignment

- **Conditions:**
 - Noise ratio < 20%.
 - Density > mean density.
- **Code:**

```
def assign_noise_to_clusters(data, labels, density, max_noise_ratio=0.2):
    noise_mask = (labels == -1)
    if np.sum(noise_mask) / len(data) > max_noise_ratio:
        return labels
    nbrs = NearestNeighbors(n_neighbors=1).fit(data[~noise_mask])
    _, indices = nbrs.kneighbors(data[noise_mask])
    labels[noise_mask] = labels[~noise_mask][indices.flatten()]
    return labels
```

2.7 Evaluation

- **Metric:** Silhouette Coefficient.
- **Code:**

```
silhouette_avg = silhouette_score(data, dpc_labels)
```

3 Key Improvements Summary

Table 1: Comparison of Original vs. Improved DPC

Step	Original DPC	Improved DPC
Density	Fixed d_c cutoff	KNN-adaptive
Centers	Manual selection	Automated via γ_i knee detection
Assignment	Nearest neighbor	K-means initialization
Noise	Permanent halo	Conditional reassignment
Validation	None	Silhouette score

Appendix

Algorithm 1 Improved Density Peaks Clustering (DPC)

```

1: Input: Data matrix  $\mathbf{X} \in \mathbb{R}^{n \times m}$ ,  $k$  (KNN neighbors), target_range =  $(k_{\min}, k_{\max})$ 
2: Output: Cluster labels  $\mathbf{y}$ , optimal cluster count  $k^*$ 
3: procedure COMPUTEDENSITY( $\mathbf{X}, k$ )
4:   for each point  $\mathbf{x}_i \in \mathbf{X}$  do
5:      $d_{\text{KNN}} \leftarrow$  distances to  $k$  nearest neighbors of  $\mathbf{x}_i$ 
6:      $\rho_i \leftarrow 1/(\text{mean}(d_{\text{KNN}}) + \epsilon)$  ▷ KNN-based density
7:   end for return  $\rho$ 
8: end procedure
9: procedure COMPUTEDELTA( $\mathbf{X}, \rho$ )
10:   $\mathbf{D} \leftarrow$  pairwise_distances( $\mathbf{X}$ )
11:  for each point  $\mathbf{x}_i \in \mathbf{X}$  do
12:    if  $\exists j$  where  $\rho_j > \rho_i$  then
13:       $\delta_i \leftarrow \min(\mathbf{D}_{i,j} \mid \rho_j > \rho_i)$ 
14:    else
15:       $\delta_i \leftarrow \max(\mathbf{D}_{i,:})$  ▷ Global max distance
16:    end if
17:  end for return  $\delta$ 
18: end procedure
19: procedure CLUSTERDPC( $\mathbf{X}, k, \text{target\_range}$ )
20:   $\rho \leftarrow$  ComputeDensity( $\mathbf{X}, k$ )
21:   $\delta \leftarrow$  ComputeDelta( $\mathbf{X}, \rho$ )
22:   $\gamma \leftarrow \rho \odot \delta$  ▷ Element-wise product
23:   $k^* \leftarrow$  KneeLocator(sorted( $\gamma$ ))
24:   $k^* \leftarrow \text{clamp}(k^*, \text{target\_range})$ 
25:  centers  $\leftarrow$  top  $k^*$  indices in  $\gamma$ 
26:   $\mathbf{y} \leftarrow$  KMeans( $k^*$ , init =  $\mathbf{X}[\text{centers}]$ ).fit_predict( $\mathbf{X}$ )
27:   $\mathbf{y} \leftarrow$  ReassignNoise( $\mathbf{X}, \mathbf{y}, \rho$ )
28:  return  $\mathbf{y}, k^*$ 
29: end procedure

```
